Machine Learning Approach to Network Intrusion Detection for IoT Devices
Proposed Methodology

Group 02 – CSE499A: Khondokar Sajid (2211954042), Kazi Safin Arafat (2211778642), Moushumi Akter Mow (2021983642), and Rakibul Hasan Ridoy (1731339042)

A. Data and Preprocessing

We will use three publicly available network intrusion datasets: **NSL-KDD**, **UNSW-NB15**, and **TON-IoT**, each selected for different experimental purposes.

1. Dataset Selection

NSL-KDD: Used for baseline testing and model debugging.
UNSW-NB15: Provides modern network features suitable for ML benchmarking.
TON-IoT: Contains real IoT network traffic and attack patterns, serving as the primary dataset for training and evaluation.

2. Preprocessing Steps

Feature Encoding: Convert categorical attributes (e.g., protocol type, flag) using *Label Encoding*.
Normalization: Scale numerical features (e.g., packet size, duration) using *Min–Max Scaling*.
Data Cleaning: Remove duplicates, handle missing values, and ensure consistent feature sets.
Class Balancing: Apply *undersampling* or *SMOTE* to address class imbalance.
Edge Simulation: Reduce dataset size to 10–20% to replicate memory constraints of IoT devices.

B. Machine Learning Models for Detection

The proposed Intrusion Detection System (IDS) adopts a **hybrid machine learning architecture**, combining three complementary models to enhance detection accuracy and stability on IoT data.

1. Autoencoder (AE)

An unsupervised neural model trained on normal network behavior.
Reconstructs input data; a high reconstruction error indicates an anomaly.
Designed with 3–5 layers and optimized through **8-bit quantization** for IoT deployment.

2. Isolation Forest (IF)

A tree-based anomaly detection model that isolates abnormal data points.
Efficient and lightweight, making it well-suited for real-time detection.

3. One-Class SVM (OCSVM)

Learns a decision boundary around normal samples in feature space.
Flags any data outside this boundary as potential intrusions.

4. Model Fusion

Each model independently classifies samples as normal or anomalous.
The final decision is made through **majority voting**, balancing accuracy and computational cost.

C. Model Architecture and Optimization

1. Hybrid Pipeline Structure

Input Features → Autoencoder → Isolation Forest → One-Class SVM → Majority Voting → Final Classification

2. Optimization for IoT Devices

Pruning: Remove 30–40% of weak or redundant model parameters.
Quantization: Convert model weights from 32-bit to 8-bit precision.

Batch Inference: Process multiple samples together to minimize delay.
Performance Targets: Achieve model size <10 MB and inference latency <500 ms on Raspberry Pi or ESP32 hardware.

3. Deployment Framework

Convert all models into **TensorFlow Lite (TFLite)** format for lightweight execution.
Deploy and test the final hybrid pipeline on **Raspberry Pi 4** and **ESP32** devices.

D. Evaluation Metrics

The system will be evaluated based on both **detection performance** and **hardware efficiency**.

Category	**Metrics**
Detection Performance	Precision, Recall, F1-Score, Accuracy, False Positive Rate (FPR), ROC-AUC
Efficiency (IoT Metrics)	Inference Time (Latency, ms), Model Size (MB), Energy Consumption (mW), Memory Usage (MB)
Generalization	Cross-dataset validation between UNSW-NB15 and TON-IoT datasets

---

E. Baseline Comparison

For performance benchmarking, the proposed hybrid model (**AE + IF + OCSVM**) will be compared against traditional ML algorithms:

Random Forest (RF)
Support Vector Machine (SVM)
Logistic Regression (LR)

This comparison will demonstrate the advantages of our hybrid approach in achieving higher accuracy, lower false-positive rates, and better efficiency on IoT hardware.

F. Expected Outcomes

A **lightweight hybrid IDS** capable of local deployment on IoT devices.
Higher detection accuracy with reduced false positives compared to baseline models.
Optimized model footprint under 10 MB through pruning and quantization.
Real-time intrusion detection with inference latency below 500 ms.
Elimination of **cloud dependency**, ensuring faster detection and improved privacy.